

```

import time

def linear_search(arr, target):
    for i in range(len(arr)):
        if arr[i] == target:
            return i
    return -1

# Input
arr = list(map(int, input("Enter the elements of the list: ").split()))
target = int(input("Enter the element to search: "))

# Measure execution time
start_time = time.perf_counter()

result = linear_search(arr, target)

end_time = time.perf_counter()

# Output
if result != -1:
    print(f"Element found at index {result}")
else:
    print("Element not found")

execution_time = end_time - start_time
print(f"Execution Time: {execution_time:.10f} seconds")

# Complexity
print("\nTime Complexity:")
print("Best Case      : O(1)")
print("Average Case   : O(n)")
print("Worst Case      : O(n)")

print("\nSpace Complexity:")
print("O(1)")

```

Enter the elements of the list: 23 67 34 65 12
Enter the element to search: 67
Element found at index 1
Execution Time: 0.0000965340 seconds

Time Complexity:
Best Case : O(1)
Average Case : O(n)
Worst Case : O(n)

Space Complexity:
O(1)

```

import time

def binary_search(arr, key):
    low = 0
    high = len(arr) - 1

    while low <= high:
        mid = (low + high) // 2

        if arr[mid] == key:
            return mid
        elif arr[mid] < key:
            low = mid + 1
        else:
            high = mid - 1

    return -1

# Input
arr = list(map(int, input("Enter sorted elements: ").split()))
key = int(input("Enter the element to search: "))

# Execution Time Start
start_time = time.perf_counter()

```

```
result = binary_search(arr, key)

# Execution Time End
end_time = time.perf_counter()

# Output
if result != -1:
    print("Element found at index", result)
else:
    print("Element not found")

print("Execution Time:", end_time - start_time, "seconds")

# Complexity
print("\nTime Complexity:")
print("Best Case      : O(1)")
print("Average Case   : O(log n)")
print("Worst Case     : O(log n)")

print("\nSpace Complexity:")
print("O(1)")
```

```
Enter sorted elements: 5 10 15 20 25 30 35
Enter the element to search: 25
Element found at index 4
Execution Time: 0.00016338299997187278 seconds
```

```
Time Complexity:
Best Case      : O(1)
Average Case   : O(log n)
Worst Case     : O(log n)
```

```
Space Complexity:
O(1)
```